Proof of Delivery (POD) — System Design Document

Table of Contents

1. Problem Statement

Right now, delivery drivers carry paper forms. They write down the AWB number, collect a signature, and physically file those sheets at the end of the day. This process breaks down in several ways:

- Records get lost or damaged in the field
- The operations team has zero real-time visibility
- Disputes are hard to resolve with no photo proof
- End-of-day reconciliation is slow and error-prone

The goal is to replace this entirely with a simple mobile app that a driver can use with one hand while holding a package with the other.

2. Proposed Solution

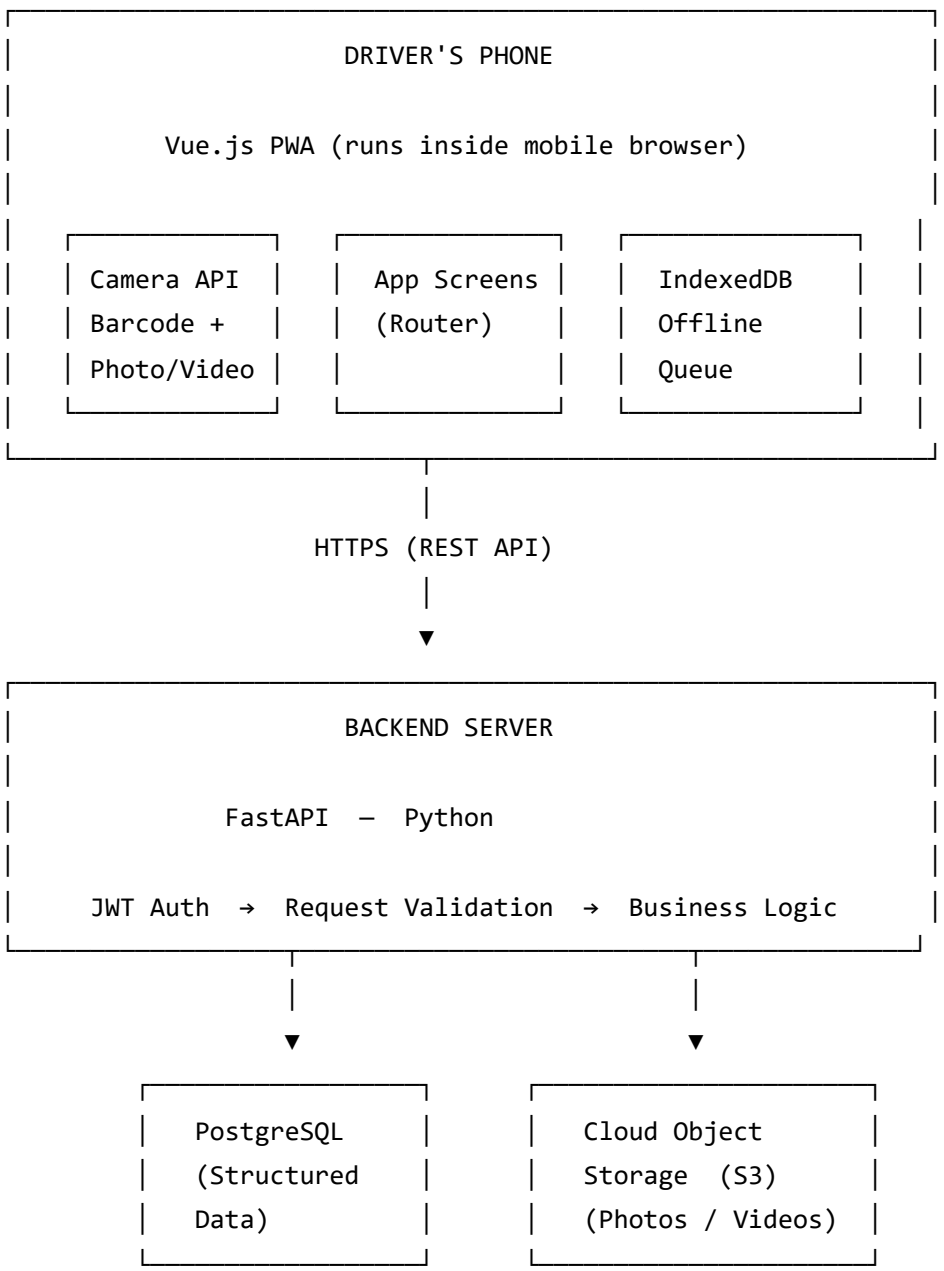
A **Progressive Web App (PWA)** — it runs in the phone's browser, can be installed on the home screen like a native app, and works partially even without an internet connection. No app store needed, no installation headaches across a fleet.

The driver's entire job in the app comes down to three steps:

1. **Scan** the barcode or QR code on the package to read the AWB number
2. **Capture** a photo or short video as visual proof
3. **Submit** — the app handles the rest

Total time per delivery: under 60 seconds.

3. System Architecture Overview



The Vue app lives on the driver's phone. Every action — scan, capture, submit — flows to the FastAPI server over HTTPS. The server splits responsibility clearly: structured data (who delivered what, when, where) goes into PostgreSQL, and the actual media files (photos and videos) go into cloud object storage like AWS S3. The database only holds a URL pointing to where the file lives, not the file itself.

4. Technology Choices & Why

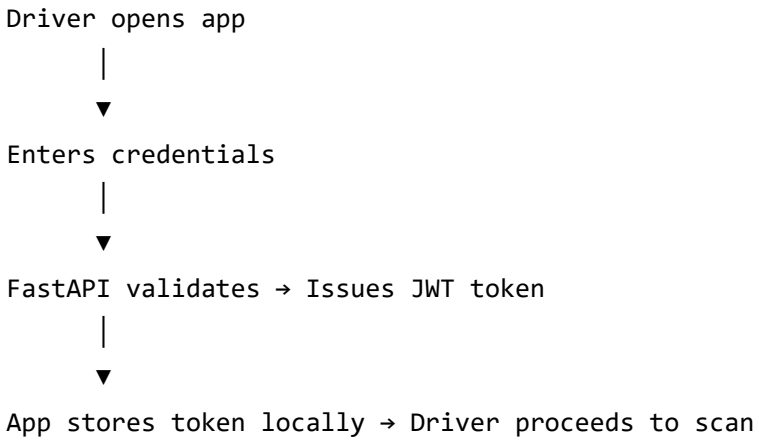
Layer	Technology	Reason
Frontend	Vue.js 3	Lightweight, fast to build, excellent PWA ecosystem
PWA capability	Vite PWA Plugin	Handles service worker and offline mode with minimal setup
Barcode Scanning	ZXing-js	Browser-native barcode/QR reader, no native app required
Media Capture	Browser MediaDevices API	Built into every modern mobile browser, no plugins
Backend	FastAPI (Python)	Async, auto-generates API docs, clean and readable
Database	PostgreSQL	Rock-solid for relational data, great for reporting later
Object Storage	AWS S3 (or Cloudflare R2)	Purpose-built for media files, cheap at scale
Authentication	JWT Tokens	Stateless — works cleanly with mobile clients

> **Why a PWA and not a native app?**
> A PWA installs from a browser link. Every driver in a fleet of 200 gets it by opening one URL — no Play Store approvals, no MDM configuration, no versioning across devices. For a logistics company rolling this out quickly, that matters a lot.

5. Core Workflows

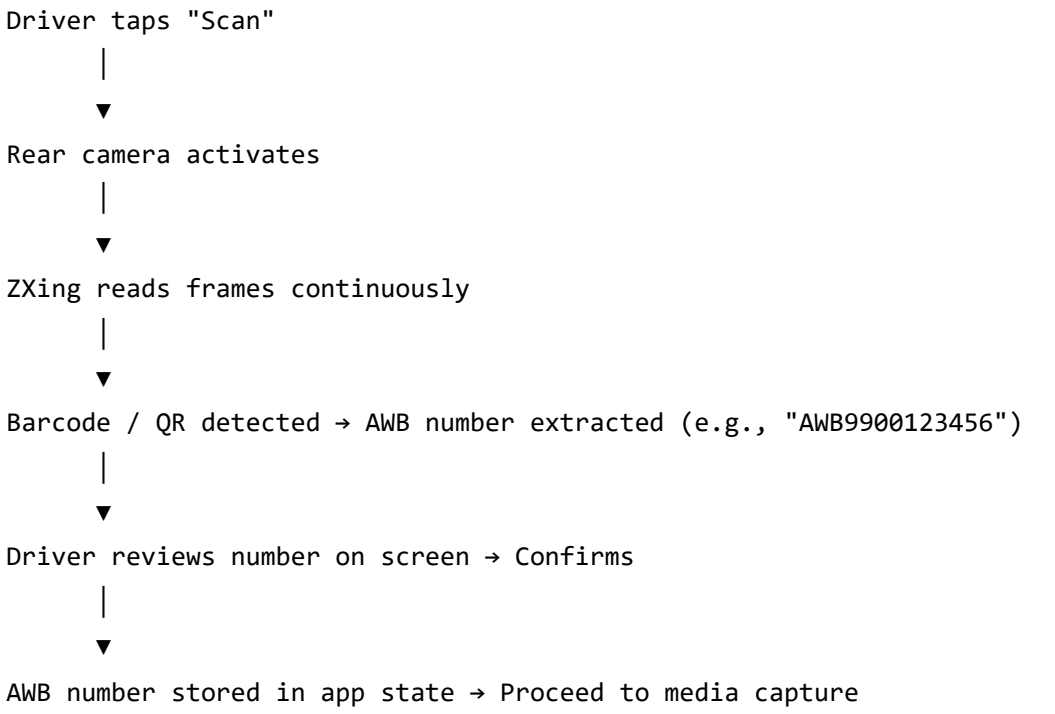
5.1 — Driver Login

The driver opens the app and enters their credentials. The backend verifies them and returns a JWT token. All future requests from that device carry this token to prove identity. The token is valid for one shift (8 hours), after which a silent refresh happens in the background.



5.2 — AWB Scanning Workflow

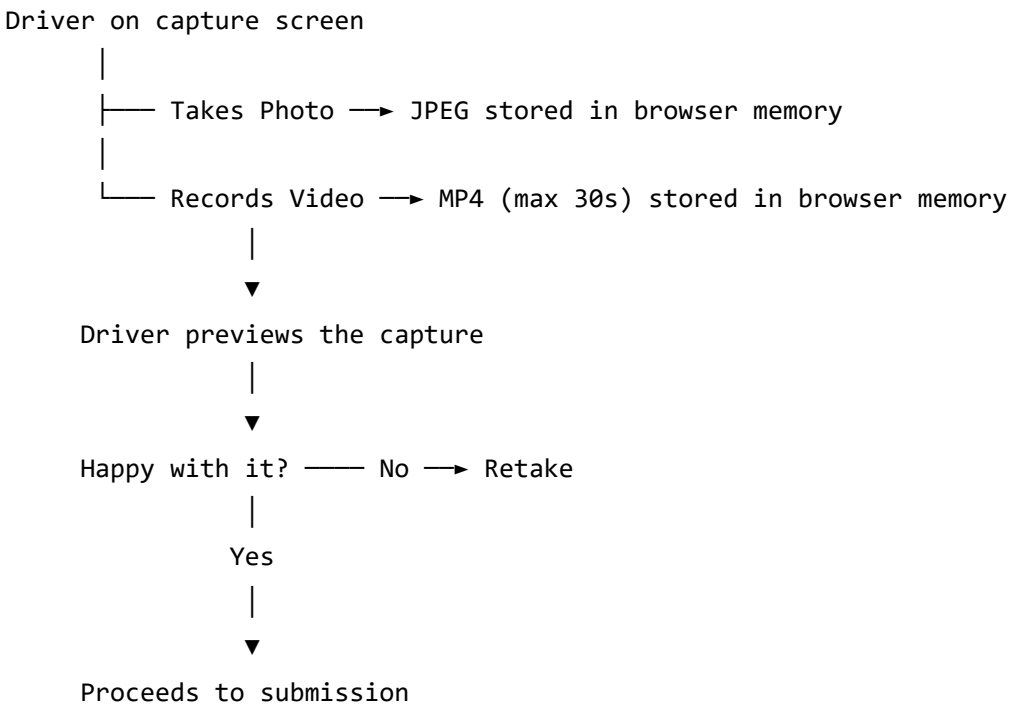
The driver taps "Scan Package." The app activates the rear camera. The ZXing library reads the camera feed frame by frame. The moment it detects a valid barcode or QR code, it extracts the text (the AWB number) and stops the camera. The driver sees the number on screen, confirms it, and moves on.



No manual typing. No misreads from handwriting. If the scan fails, the driver can type the number manually as a fallback.

5.3 — Media Capture Workflow

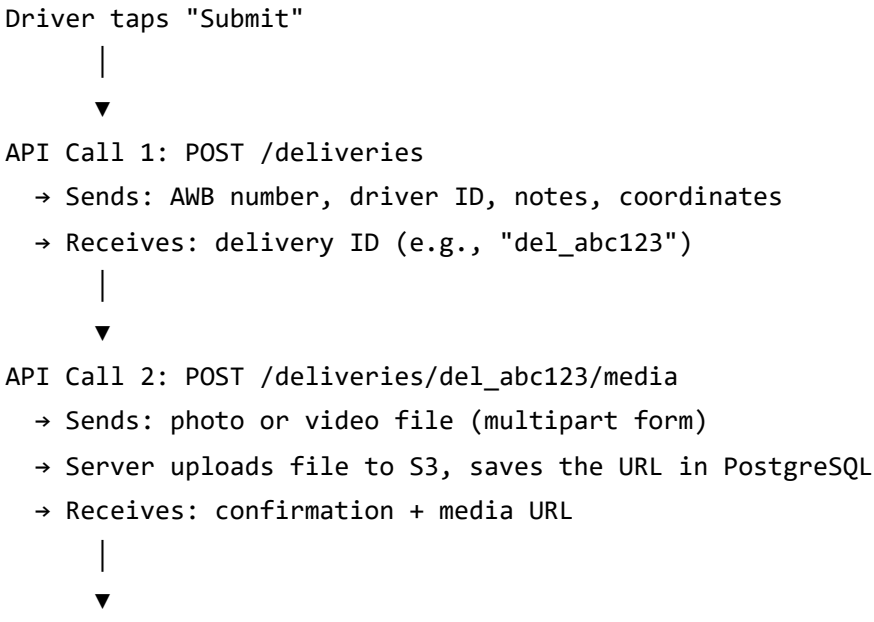
After confirming the AWB, the driver is taken to the capture screen. They choose to take a photo or record a short video (max 30 seconds). The captured file is held in memory temporarily — it is not uploaded yet.



5.4 — Submission Workflow

The driver hits "Submit Delivery." The app makes two sequential API calls:

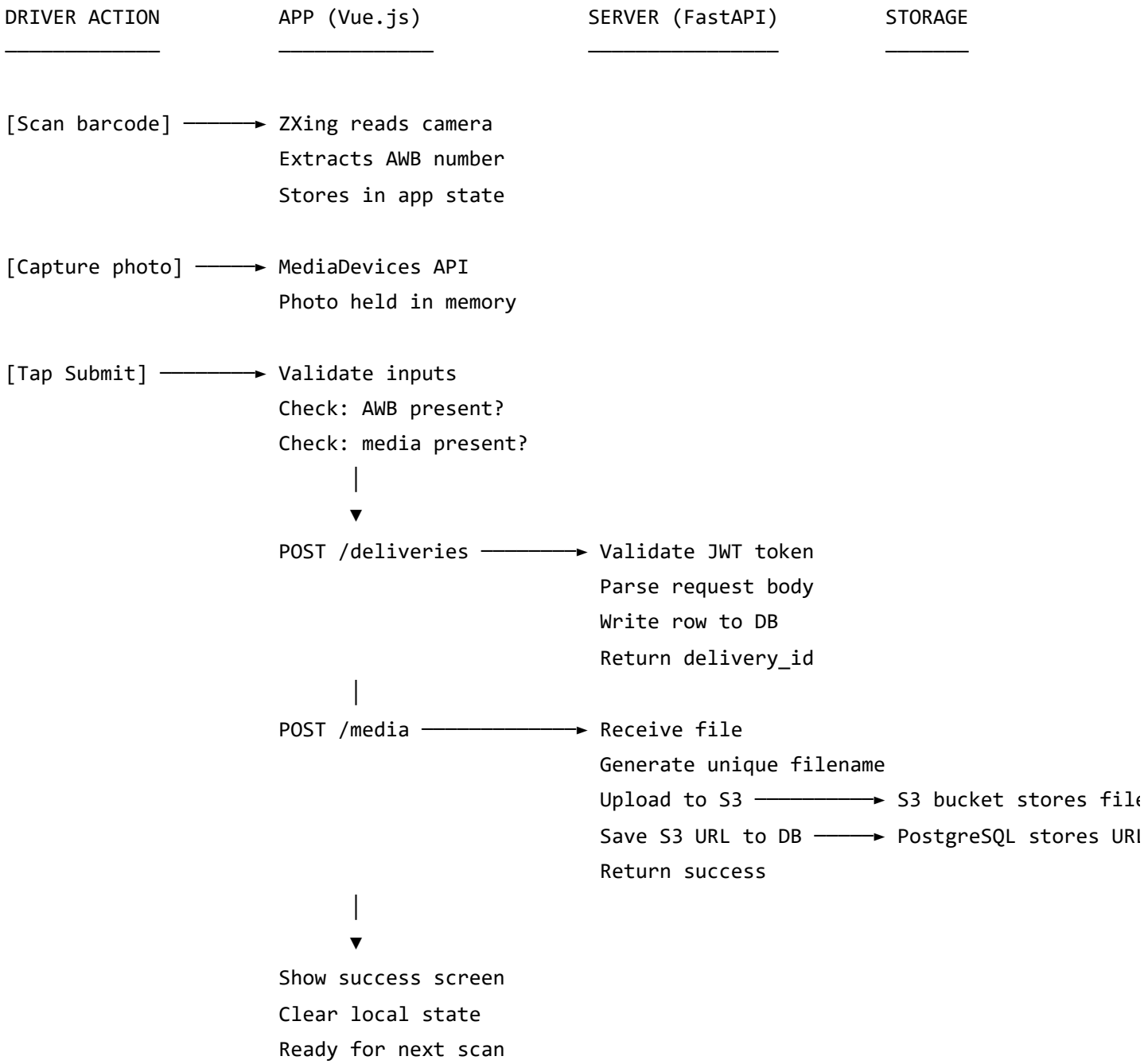
1. First call — create the delivery record (AWB number, driver ID, timestamp, GPS coordinates if available, any notes)
2. Second call — upload the media file, referencing the delivery ID from step 1



6. Data Flow

6.1 — Full End-to-End Data Flow

This diagram shows the complete journey of a single delivery, from the driver's tap to the record being stored.



6.2 — Media Storage Flow (Closer Look)

A common question is: why not store photos in the database directly?

The answer is that databases are optimized for small, structured records — rows of text and numbers. Storing binary files (photos, videos) in a database makes it slow and expensive very quickly. The standard approach is to store the file in object storage (S3) and store only the URL in the database.

```
Photo file (e.g., 3MB JPEG)
        │
        ▼
FastAPI receives it as multipart upload
        │
        ▼
Server generates a safe, unique filename:
   "AWB9900123456_driver42_20250607_1045.jpg"
        │
        ▼
File uploaded to S3 bucket under path:
   /media/AWB9900123456/filename.jpg
        │
        ▼
S3 returns the storage URL
        │
        ▼
URL saved in PostgreSQL's media_files table:
   storage_url = "https://s3.amazonaws.com/pod-bucket/media/AWB9900123456/..."
        │
        ▼
Any future lookup for AWB9900123456
→ Queries PostgreSQL
→ Gets back the S3 URL
→ Client fetches photo directly from S3
```

6.3 — Read Flow (Ops Team Lookup)

When a supervisor or operations team member looks up a delivery by AWB number:

```
Supervisor searches AWB9900123456
        |
        ▼
GET /deliveries?awb=AWB9900123456
        |
        ▼
FastAPI queries PostgreSQL
→ Returns: driver name, timestamp, GPS, notes, status
→ Returns: list of media URLs from media_files table
        |
        ▼
Frontend displays delivery record
→ Photo/video loaded directly from S3 URL
        |
        ▼
Supervisor can confirm delivery was completed
```

7. Data Model

Three tables cover everything this application needs.

`users` — Drivers and Admin Staff

Stores everyone who has an account in the system. The `role` field separates drivers from admins/supervisors.

Column	Type	Description
id	UUID	Unique identifier, auto-generated
name	VARCHAR	Full name of the driver
email	VARCHAR	Login email, must be unique
password	VARCHAR	Stored as a bcrypt hash, never plain text
role	VARCHAR	Either `driver` or `admin`
is_active	BOOLEAN	Can be deactivated without deleting
created_at	TIMESTAMP	When the account was created

`deliveries` — The Core Record

One row is created for each delivery attempt. This is the heart of the system.

Column	Type	Description
id	UUID	Unique delivery record ID
awb_number	VARCHAR	The scanned AWB, indexed for fast lookup
driver_id	UUID	Foreign key → `users.id`
status	VARCHAR	`pending` , `delivered` , or `failed`
recipient_name	VARCHAR	Name of the person who received it
notes	TEXT	Any free-text notes the driver added
latitude	DECIMAL	GPS latitude at time of delivery
longitude	DECIMAL	GPS longitude at time of delivery
delivered_at	TIMESTAMP	When the driver submitted the proof
created_at	TIMESTAMP	When the record was first created

> The `awb_number` column is indexed. This means even with millions of records, looking up a delivery by AWB is near-instant.

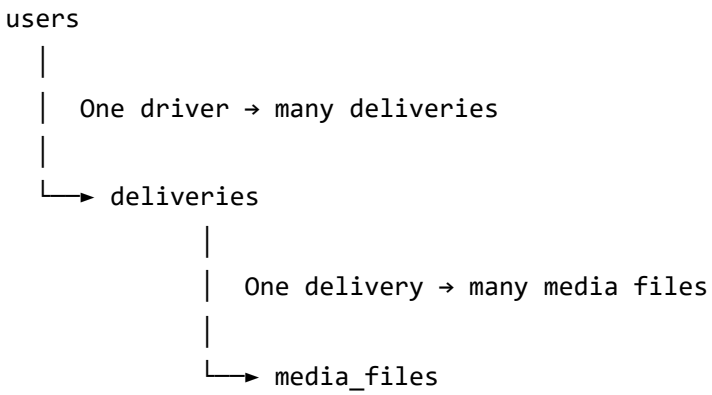
`media_files` — Photos and Videos

One delivery can have multiple media files (e.g., one photo of the package, one of the door). Each file is a separate row linked back to its delivery.

Column	Type	Description
id	UUID	Unique file record ID
delivery_id	UUID	Foreign key → `deliveries.id`
file_type	VARCHAR	Either `photo` or `video`
storage_url	TEXT	Full URL of the file in S3
file_size_kb	INTEGER	Size of the file, useful for monitoring

Column	Type	Description
uploaded_at	TIMESTAMP	When it was uploaded to storage

Entity Relationship



8. API Design

The API follows REST conventions. Every request must carry a valid JWT token in the Authorization header.

Authentication

Method	Endpoint	Purpose
POST	`/auth/login`	Driver submits credentials, receives JWT token
POST	`/auth/refresh`	Exchange an expiring token for a new one silently

Deliveries

Method	Endpoint	Purpose
POST	`/deliveries`	Create a new delivery record for a scanned AWB
GET	`/deliveries/{id}`	Get full details of one delivery
GET	`/deliveries?awb={awb}`	Look up a delivery by AWB number
GET	`/deliveries/mine`	Driver fetches their own delivery history

Method	Endpoint	Purpose
PATCH	`/deliveries/{id}/status`	Update status to `delivered` or `failed`

Media

Method	Endpoint	Purpose
POST	`/deliveries/{id}/media`	Upload a photo or video for a delivery
GET	`/deliveries/{id}/media`	List all media files attached to a delivery

Example: Create a Delivery

Request

```
POST /deliveries
Authorization: Bearer <token>

{
  "awb_number": "AWB9900123456",
  "recipient_name": "Sarah Ahmed",
  "notes": "Left with building reception",
  "latitude": 26.7271,
  "longitude": 88.3953
}
```

Response

```
{
  "id": "del_abc123",
  "awb_number": "AWB9900123456",
  "status": "pending",
  "delivered_at": null,
  "created_at": "2025-06-07T10:30:00Z"
}
```

Example: Upload Media

Request

```
POST /deliveries/del_abc123/media
Authorization: Bearer <token>
Content-Type: multipart/form-data

file: [binary image data]
file_type: "photo"
```

Response

```
{
  "id": "med_xyz789",
  "delivery_id": "del_abc123",
  "file_type": "photo",
  "storage_url": "https://pod-bucket.s3.amazonaws.com/media/AWB9900123456/del_abc123_photo.jpg",
  "uploaded_at": "2025-06-07T10:31:00Z"
}
```

9. Offline Handling

This is one of the more interesting challenges for a field app. Drivers work in basements, rural areas, and buildings with poor signal. The app cannot assume a live internet connection at the moment of delivery.

How it works

When the driver taps "Submit" and there is no connection:

1. The app detects the failure (or the lack of network before even trying)
2. The delivery data and media file are saved to **IndexedDB** — a storage system built into every browser
3. The driver sees a "Saved offline, will upload when back online" message
4. A **service worker** runs in the background and watches for the connection to return
5. The moment connectivity is restored, it automatically replays the queued submissions in order

From the driver's perspective, they just keep working. The queue handles itself.

```
No signal at delivery point
        │
        ▼
App saves to local IndexedDB queue:
  { awb: "AWB123", media: [file], notes: "...", timestamp: "..." }
        │
        ▼
Driver continues to next package
        │
        ▼
Connection restored (back on the road / in office)
        │
        ▼
Service worker detects online event
        │
        ▼
Replays queue: submits each saved delivery in order
        │
        ▼
All records synced to server ✓
```

10. Security Considerations

Authentication and Authorization

Every API request requires a valid JWT token. The token contains the driver's ID and role. On the backend, every endpoint checks that the requesting driver is only accessing their own data — a driver cannot see another driver's history.

Passwords

Passwords are always stored as bcrypt hashes. The actual password is never stored anywhere.

File Uploads

The server validates every uploaded file before passing it to S3. Only `image/jpeg`, `image/png`, and `video/mp4` are accepted. File size is capped at 50MB. The server always renames uploaded files — a user can never control the filename stored in S3.

Media Access

The S3 bucket is private. Media files are never publicly accessible via a raw URL. Instead, the backend generates a **pre-signed URL** on demand — a temporary link that expires after 24 hours. This means even if someone gets hold of a media URL, it stops working within a day.

Data Integrity

The AWB number and driver ID are always server-side validated and matched. A driver cannot submit a media file for a delivery that doesn't belong to them.

📬 Contact

Designed by Shib Kumar Saraf

For inquiries or contributions, please reach out via.

📧 **Email:** shibkumarsaraf05@gmail.com

🐙 **GitHub:** github.com/shib1111111